

Introduction

This task is a classical crackme challenge. You are given a Portable Executable (ELF), which asks you for a password. You will need to apply different reversing techniques which were introduced the lecture. The challenge consists of three stages with increasing difficulty.

Helpful tools

- gdb / edb
- strings
- ltrace
- strace
- ghidra

Exercise 1: CrackMe

Download the crackme from OLAT into a linux environment (recommended is Kali). You can execute the file to test it. Use the tools listed above, to find the passwords. We don't want you to use ghidra for this exercise, so limit yourself to the other tools. For the report, briefly explain how you found these passwords.

Exercise 2: Reversing

As a Security Analyst at *Gotham Security Inc.*, a fictional penetration testing company, you are currently involved in an ongoing penetration test for a client. During the course of the test, your colleagues have identified a binary that uses a *homemade* encryption algorithm for communicating some confidential information within the company. This particular binary is within the scope of the ongoing penetration test, and you have been assigned the task of reverse engineering it. Your primary objective is to analyze the encryption algorithm used by the binary and evaluate its security posture.

Furthermore, your team has also discovered some notes that were found alongside the binary. These notes suggest that there was a slight change made to the encryption algorithm prior to your team intercepting the binary. The change was related to the removal of a *time syncing mechanism*, which was found to be cumbersome for the company employees to use.

Your task is to thoroughly examine the algorithm and reconstruct the steps involved while considering the changes made to the removal of the *time syncing mechanism* and the addition of a different step. Fortunately, you have been provided with the change to be made in the form of an encoded string (*secret2.txt*). It will be your responsibility to decode the string and incorporate the changes into the reconstructed algorithm.

Note: If you face any difficulty during the course of this exercise feel free to contact Daniel Reti.

Challenge files:

- ***communicate_secret***: Binary to analyze and reverse engineer.
- ***secret1.txt***: the text you will be using as an argument to encrypt in your reconstructed program.
- ***secret2.txt***: the change that is needed to be made to the algorithm. This change should reflect into your reconstructed program.

Task Description:

1. Decode the encoded text in *secret1.txt* and *secret2.txt* to reveal their original text.
2. Reverse engineer the working of the binary, including all seven subcalculations, and reconstruct a similar algorithm as the original program. This reconstructed program must incorporate the specified change given in the encoded string (*secret2.txt*) and must be equivalent to the original program.

Hints and Directions

- You can use any debugger or disassembler of your choice to complete this task. While some sub-calculations can be solved by observing debugging outputs, others may require you to dive deeper into disassembly and debugging.
- For disassembly and debugging, you may use GDB: The GNU Project Debugger. To analyze the binary, you will need to set breakpoints at the functions corresponding to sub-calculations (e.g., `calculate_1`) and then step through the instructions to determine the operations they are performing. You can refer to the following GDB cheatsheets for basic gdb operations: `gdb-cheatsheet-1` or `gdb-cheatsheet-2`.
In this exercise you can also use ghidra, though ghidra alone might not be enough to solve the exercise.
- You might encounter some helper functions like `strchr` while debugging. You need not explain the behaviour of the `strchr` function, as it is just a helper function. Here is a useful reference to find such functions and their descriptions [Standard C Library Functions Table](#).
- Your main focus should be on the functions which begin with "calculate". Your reverse-engineered code does not need to print debugging outputs or perform input validation.
- The final output will be a number that corresponds to the second-to-last debugging output, in the same number representation. The final flag will be `FLAG{x}`, where `x` is the final output of your reversed program.

Submission Guidelines:

Please make sure your submission includes the following:

1. A flag submission.
2. The secret strings that are included in both the text files (*secret1.txt* and *secret2.txt*), along with a brief description of the encoding used on them.

3. For each of the computations of the reverse binary (*communicate_secret*), you must provide a short explanation of what they are doing and how you were able to figure it out.
4. The reconstructed algorithm as source code. For this exercise, you are free to use any programming or scripting language to reconstruct the program.

Report

Write a brief report including your solutions and your findings. Then, submit the report via Olat. You can find a $\text{\LaTeX}$ template on OLAT.